ASSIGNMENT 3: DATABASE DESIGN ANALYSIS

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: KUNGUMA KEERTHANA M

REGISTER NO: 2511021240051

COURSE: BCA

YEAR: 2ND YEAR

PROJECT NAME: Online Food Delivery System

1. Aim

To analyze the database design of the Online Food Delivery System by identifying data redundancy, insertion, deletion and update anomalies, functional dependencies, and suitable improvements.

2. Introduction

The Online Food Delivery System is a database-based application that allows customers to browse restaurants, select food items, place orders, make payments, and track deliveries.

The database consists of ten tables: Customer, Address, Restaurant, Food_Item, Category, Orders, Order_Item, Payment, Delivery_Partner, and Delivery.

A good database design reduces duplicate data, maintains consistency, and makes data storage and retrieval easier.

3. Data Redundancy

Data redundancy means storing the same information unnecessarily in multiple places.

Possible redundancy in the Online Food Delivery System includes:

- **Customer details:** If customer names, phone numbers, and addresses are repeatedly stored in order records, the same information may appear many times.
- **Restaurant details:** Restaurant names and phone numbers may be duplicated if they are stored directly in every food item or order record.
- **Category details:** Category names may be repeated for every food item if category information is not stored separately.
- **Food item details:** Food names and prices may be repeated in order records instead of referencing the Food_Item table.

The current design reduces these problems by separating customer, restaurant, category, and food item information into different tables.

4. Functional Dependencies

A functional dependency exists when one attribute uniquely determines another attribute.

The functional dependencies in the proposed database are:

Table	Functional dependency
Customer	Customer_ID → Name, Email, Phone, Password
Address	Address_ID → Customer_ID, House_No, Street, City, Pincode
Restaurant	Restaurant_ID → Restaurant_Name, Phone, Location, Rating
Category	Category_ID → Category_Name
Food_Item	Food_ID → Restaurant_ID, Category_ID, Food_Name, Price, Availability
Orders	Order_ID → Customer_ID, Address_ID, Order_Date, Total_Amount, Order_Status
Order_Item	Order_Item_ID → Order_ID, Food_ID, Quantity, Subtotal
Payment	Payment_ID → Order_ID, Payment_Method, Payment_Date, Amount, Payment_Status
Delivery_Partner	Partner_ID → Partner_Name, Phone, Vehicle_No, Status
Delivery	Delivery_ID → Order_ID, Partner_ID, Pickup_Time, Delivery_Time, Delivery_Status

Note: These dependencies assume that the listed IDs are unique primary keys. Additional dependencies may exist if the application enforces other uniqueness rules.

5. Insertion, Deletion, and Update Anomalies

5.1 Insertion Anomaly

An insertion anomaly occurs when information cannot be added without entering unrelated information.

Example: If restaurant details and food item details were stored in one table, adding a new restaurant without any food items could be difficult because a food item record might be required.

Solution: Store restaurant information separately in the Restaurant table and food information in the Food_Item table.

5.2 Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes other important information.

Example: If restaurant information were stored only alongside its food items, deleting the restaurant's last food item could also remove the restaurant's details.

Solution: Maintain separate Restaurant and Food_Item tables so that deleting a food item does not automatically delete restaurant information.

5.3 Update Anomaly

An update anomaly occurs when the same information is stored in multiple places and every copy must be updated.

Example: If a restaurant's phone number appeared in several food item records, changing the phone number in only some records would result in inconsistent data.

Solution: Store the restaurant's phone number only in the Restaurant table and refer to the restaurant through Restaurant_ID.

6. How Poor Database Design Causes Anomalies

Poor database design combines unrelated information in the same table or stores the same facts repeatedly. This creates several problems:

- **Data inconsistency:** Different records may contain different versions of the same information.
- **Wasted storage:** Repeated information occupies unnecessary space.
- **Difficult insertion:** New information may require unrelated data.
- **Accidental data loss:** Deleting a record may remove information that should be retained.
- **Difficult updates:** Multiple records may need to be changed to maintain consistency.

These problems can affect the reliability and efficiency of the Online Food Delivery System.

7. Proposed Improvements

The following improvements can make the database more reliable:

1. **Apply normalization:** Organize the tables to reduce redundancy and avoid insertion, deletion, and update anomalies.
2. **Use primary keys:** Assign unique IDs to identify each record in every table.
3. **Use foreign keys:** Connect related tables and maintain referential integrity.
4. **Separate entities:** Keep customer, restaurant, category, food item, payment, and delivery partner details in their respective tables.
5. **Improve the Order_Item table:** Use Order_ID and Food_ID as a composite primary key, or retain Order_Item_ID and add a unique constraint on Order_ID and Food_ID if duplicate food lines are not allowed.
6. **Maintain order prices:** Store the price charged at the time of ordering in Order_Item so that historical order totals remain accurate even when menu prices change.
7. **Validate data:** Apply suitable constraints for email uniqueness, positive quantities, valid prices, and permitted order and payment statuses.
8. **Protect passwords:** Store securely hashed passwords rather than plain-text passwords.
9. **Maintain consistency:** Calculate or validate Total_Amount using the order's item subtotals and any applicable additional charges or discounts.
10. **Review relationships:** Ensure that each order's delivery address belongs to the customer who placed the order.

8. Normalization

Normalization is the process of organizing database tables to reduce redundancy and improve data integrity.

- **First Normal Form (1NF):** Each field should contain a single, atomic value, without repeating groups.
- **Second Normal Form (2NF):** The table should be in 1NF, and every non-key attribute should depend on the whole candidate key.
- **Third Normal Form (3NF):** The table should be in 2NF, and non-key attributes should not depend transitively on a candidate key.

The proposed database separates information into related tables, which supports normalization. However, full compliance with each normal form must be verified by checking all functional dependencies and candidate keys.

9. Result

The database design of the Online Food Delivery System was analyzed for data redundancy, functional dependencies, and insertion, deletion, and update anomalies. Suitable improvements, including normalization, primary and foreign keys, validation constraints, and secure password storage, were proposed.

10. Conclusion

A well-designed database is essential for the effective operation of an Online Food Delivery System. Separating information into related tables reduces unnecessary duplication and helps prevent data anomalies. Applying normalization, maintaining referential integrity, and using suitable validation rules improve data consistency, reliability, and security.